

CENG 499

Introduction to Machine Learning

Spring 2024 - 2025

Programming Assignment 1 - Decision Trees

Due date: 28 03 2025, Friday, 23:59

1 Overview

In this homework, you will implement a decision tree classifier from scratch using the Gini index as the splitting criterion. The implementation will be evaluated on the Wine dataset, which you will analyze through exploratory data analysis (EDA) before proceeding. You will split the dataset into training, validation, and test sets, train the decision tree, evaluate its performance, and apply two pruning methods: reduced error pruning and a custom pruning method based on likelihood calculations assuming a normal distribution of inputs. This assignment aims to deepen your understanding of decision tree algorithms, their implementation, and the impact of pruning on model performance.

Keywords: *decision trees, gini index, pruning, wine dataset, exploratory data analysis*

2 Introduction

Decision trees are a fundamental machine learning technique that is used for classification tasks. They work by recursively splitting the feature space into regions based on feature values, aiming to minimize impurity (e.g., using the Gini index) at each split. In this homework, you will build a decision tree classifier from scratch, without relying on pre-built libraries like `scikit-learn`'s `DecisionTreeClassifier`, though you may use libraries like `numpy` and `pandas` for basic data handling and computations.

The Wine dataset, sourced from the UCI Machine Learning Repository, contains chemical analysis results of wines from three different cultivars in Italy. It includes 178 samples, each with 13 numerical features (e.g., alcohol content, malic acid, total phenols) and a target label indicating one of three classes (cultivars). Your task begins with an exploratory data analysis to understand the dataset's structure and relationships, followed by implementing the decision tree algorithm, training it, and applying pruning techniques to improve generalization.

You will:

- Perform exploratory data analysis to examine the dataset's features and their relationships with the target classes.
- Implement a decision tree using the Gini index to measure impurity and guide splits.
- Train the tree on the training set and evaluate its initial performance on the test set.
- Apply reduced error pruning using a validation set to simplify the tree and prevent overfitting.

- Implement a novel pruning method that calculates the likelihood of splits under a normal distribution assumption and prunes based on a threshold parameter.
- Compare the performance of the unpruned and pruned trees across multiple runs.

3 Tasks

3.1 Task 1: Exploratory Data Analysis

Before implementing the decision tree, you will perform an exploratory data analysis (EDA) on the Wine dataset to gain insights into its structure and characteristics. Load the dataset using `scikit-learn`'s `load_wine()` function or `pandas` from a CSV file. Your EDA should include the following steps:

- **Display the First Five Rows:** Print the first five rows of the dataset to understand its format, including feature values and class labels. This helps verify data loading and provides an initial look at the data.
- **Calculate Summary Statistics:** Compute and display summary statistics (e.g., mean, standard deviation, min, max) for each feature. This reveals the scale, spread, and central tendency of the data, which are crucial for interpreting feature importance in the decision tree.
- **Visualize Feature Relationships:** Select two-thirds of the features (i.e., 9 out of 13, rounded up from 8.67) and create pair plots (scatterplot matrices) to visualize their pairwise relationships and how they correlate with the three classes. Use distinct colors or markers for each class to highlight class separability. This step helps identify which features might be most discriminative for the decision tree.

Include these results in your report to provide context for your implementation.

3.2 Task 2: Implement Decision Tree

Implement a decision tree classifier from scratch using the Gini index as the impurity measure for splitting nodes. The Gini index quantifies the impurity of a node by calculating the probability of incorrectly classifying a randomly chosen sample if it were labelled according to the distribution of classes in that node. You are prohibited from using library functions that directly implement decision trees (e.g., `scikit-learn`'s `DecisionTreeClassifier`), but you may use library functions for numerical computations and libraries such as `pandas` for data manipulation.

Your implementation should include the following.

- A class `DecisionTree` with:
 - `__init__(max_depth=None, min_samples_split=2)`: Initialize the tree with optional parameters to control its depth and the minimum number of samples required to split a node.
 - `fit(X, y)`: Build the tree recursively by finding the best splits based on the Gini index and partitioning the data accordingly.
 - `predict(X)`: Traverse the tree to classify each sample in the input data `X`.
- **Recursive Tree Construction:**
 - For each node, compute the Gini index for all possible splits across all features and their unique values (or midpoints between consecutive values for efficiency).
 - Select the split that yields the lowest weighted Gini index across the resulting child nodes.

- Recursively build left and right subtrees until a stopping criterion is met (e.g., maximum depth reached, fewer samples than `min_samples_split`, or all samples belong to the same class).
- **Leaf Nodes:** Assign the majority class of the samples reaching a leaf node as its predicted label.
- **Optional Visualization:** Implement a method to print or visualize the tree structure (e.g., as text like "Feature 2 = 3.0, Left: Class 0, Right: ...") to aid debugging and understanding.

The Gini index for a node is defined as:

$$\text{Gini} = 1 - \sum_{i=1}^k p_i^2$$

where p_i is the proportion of samples of class i in the node, and k is the number of classes (here, $k = 3$ for the Wine dataset). For a split into left and right child nodes, the weighted Gini index is:

$$\text{Weighted Gini} = \frac{n_{\text{left}}}{n} \cdot \text{Gini}_{\text{left}} + \frac{n_{\text{right}}}{n} \cdot \text{Gini}_{\text{right}}$$

where n_{left} and n_{right} are the number of samples in the left and right child nodes, and n is the total number of samples at the parent node.

3.3 Task 3: Train and Evaluate

Train and evaluate your decision tree implementation as follows:

- **Dataset Split:** Divide the Wine dataset into three subsets: 60% for training (approximately 106 samples), 20% for validation (approximately 36 samples), and 20% for testing (approximately 36 samples). Use random sampling to ensure a fair split, and set a random seed (e.g., 42) for reproducibility during development.
- **Training:** Train the decision tree on the training set by calling `fit(X_train, y_train)`. Initially, use no depth limit to allow the tree to grow fully, capturing all possible splits.
- **Evaluation:** Compute the accuracy on the test set, defined as the proportion of correctly classified samples: $\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total test samples}}$. Report this as the baseline performance of the unpruned tree.
- **Optional Visualization:** If implemented, display the tree structure to verify its construction and understand the splits chosen.

3.4 Task 4: Implement Reduced Error Pruning

To prevent overfitting, implement reduced error pruning using the validation set:

- **Process:** After training the full tree on the training set, traverse the tree bottom-up (post-order). For each internal node:
 - Compute the validation accuracy with the subtree intact.
 - Temporarily replace the subtree with a leaf node labelled with the majority class of the training samples at that node.
 - Compute the validation accuracy with the subtree pruned.
 - If pruning does not decrease validation accuracy, permanently replace the subtree with the leaf; otherwise, retain the subtree.

- **Evaluation:** After pruning, evaluate the pruned tree on the test set and report the accuracy. This method aims to simplify the tree by removing branches that do not improve generalization on unseen data.

Reduced error pruning relies on the validation set to simulate unseen data, making it a practical approach to balance model complexity and performance.

3.5 Task 5: Implement Custom Pruning Method

Design and implement a custom pruning method that assesses the likelihood of splits based on a normal distribution assumption:

- **Method:** For each internal node n , let F_n be the feature used for splitting, t_n the split point, and D_n the training samples reaching that node. Assume F_n follows a normal distribution and estimate its mean and variance:

$$\mu_n = \frac{1}{|D_n|} \sum_{x \in D_n} x_{F_n}$$

$$\sigma_n^2 = \frac{1}{|D_n|} \sum_{x \in D_n} (x_{F_n} - \mu_n)^2$$

- **Likelihood Calculation:** Compute the probability that a sample goes to the left child:

$$p_{\text{left}} = P(F_n \leq t_n) = \Phi\left(\frac{t_n - \mu_n}{\sigma_n}\right)$$

where Φ is the cumulative distribution function (CDF) of the standard normal distribution (available via `scipy.stats.norm.cdf`). Then, $p_{\text{right}} = 1 - p_{\text{left}}$.

- **Pruning Criterion:** If $\min(p_{\text{left}}, p_{\text{right}}) < \alpha$ (a parameter), prune the subtree at n by replacing it with a leaf node assigned the majority class of D_n . This criterion identifies splits where one branch is unlikely, suggesting the split may be overfitting noise.
- **Implementation:** Traverse the tree post-order, applying this pruning rule. Test with at least three α values: 0.05, 0.1, and 0.2. For each α , retrain the tree on the training set, apply pruning, and report the test accuracy.

This custom method introduces a probabilistic approach to pruning, using the statistical properties of the data to refine the tree structure.

3.6 Task 6: Comparison and Discussion

Evaluate and compare the performance of your decision tree across multiple runs:

- **Multiple Runs:** Perform the entire process (Tasks 3-5) at least 5 times with different random splits of the dataset. Record the test accuracy for:
 - Unpruned tree (Task 3).
 - Reduced error pruned tree (Task 4).
 - Custom pruned tree with $\alpha = 0.05, 0.1, 0.2$ (Task 5).
- **Average Performance:** Calculate and report the average test accuracy for each method across the 5 runs. This accounts for variability due to random splitting.
- **Discussion:** In your report, analyze the results:

- How does pruning affect accuracy compared to the unpruned tree?
- Compare the effectiveness of reduced error pruning versus the custom pruning method.
- Discuss the impact of different α values on the custom pruning method: Do higher α prune more or less aggressively, and how does this affect performance?
- Reflect on challenges faced during implementation and any insights gained about decision trees or the Wine dataset.

4 Submission

Submit via ODTUClass a zip file named `pa1.zip` containing your Python source code and a report in PDF format. Include:

- **Python Code:** Your complete implementation, including EDA, decision tree, and pruning methods.
- **Report:** A PDF document with:
 - Results from Task 1 (EDA): First five rows, summary statistics, and pair plots.
 - Accuracy results for all methods across 5 runs, including averages.
 - Discussion of pruning effects and implementation challenges.

Note: Please do NOT submit your virtual environments.

5 Regulations

- **Programming Language:** Your program must be coded in Python 3. It will be tested in a virtual environment with standard libraries like (`numpy`, `pandas`, `scipy`, `matplotlib`, `seaborn`). Additional libraries can be installed if it is needed for your implementation.
- **Late Submission:** Late submissions are allowed with a penalty of $10 \times$ day. You are only allowed two days of late submission.
- **Cheating:** All submitted work must be your own. Using third-party code or large-language models (LLMs) directly (copy-paste) is considered cheating and will result in disciplinary action under our "zero tolerance" policy.
- **Newsgroup:** Follow ODTUClass for updates and discussions daily.
- **Grading:** This homework is worth 100 points and constitutes 6% of your total grade.